

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación
IC-4302 Bases de Datos II GR2

Profesor:
Kenneth Roberto Obando Rodríguez

Tarea
Práctica de Neo4J

Integrantes

King Bustamante Jenny
Ulate Obando Evelyn Vanessa

Fecha de entrega:
21 de mayo

Contents

Descripción del Ambiente	3
Evidencia de Carga de Datos	4
Consultas de Q1-Q18	5
Q1 (Media)	5
Q2 (Media)	6
Q3 (Media)	6
Q4 (Media)	7
Q5 (Media-Alta)	8
Q6 (Media-Alta)	9
Q7 (Media-Alta)	9
Q8 (Media-Alta)	10
Q9 (Difícil)	Error! Bookmark not defined.
Q10 (Difícil)	12
Q11 (Difícil)	13
Q12 (Difícil)	15
Q13 (Difícil)	16
Q14 (Difícil)	16
Q15 (Difícil)	17
Q16 (Actualización)	19
Q17 (Actualización)	20
Q18 (Análisis de Ejecución)	21
Conclusiones técnicas	28

Descripción del Ambiente

El entorno de base de datos fue desplegado de forma local mediante Docker, utilizando la imagen oficial de Neo4j versión 5.26 LTS. Esta configuración permite aislar el servicio en un contenedor reproducible, sin necesidad de instalación directa en el sistema operativo anfitrión.

```
1  services:
2    neo4j:
3      image: neo4j:5.26
4      container_name: neo4jDB
5      restart: unless-stopped
6      ports:
7        - "7474:7474"
8        - "7687:7687"
9      env_file:
10       - .env
11      volumes:
12       - neo4j_data:/data
13
14  volumes:
15    neo4j_data:
```

Docker-Compose.yml

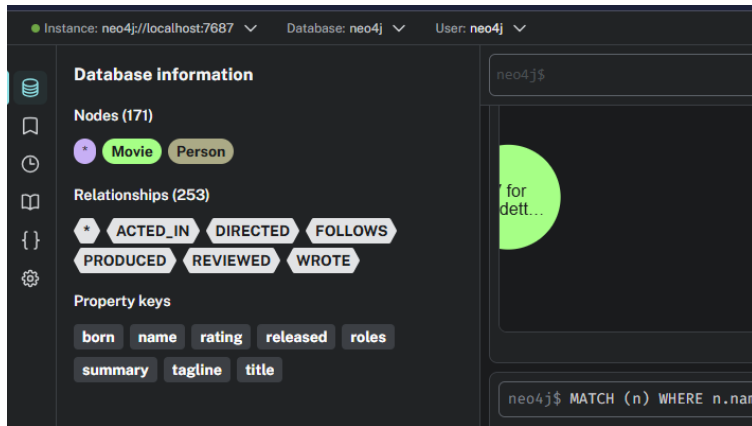
Las variables de configuración, como credenciales y parámetros de memoria, fueron gestionadas mediante un archivo de entorno externo (.env).

```
1  # credenciales
2  NEO4J_AUTH=neo4j/tu_password_aqui
3
4  # memoria
5  NEO4J_dbms_memory_heap_initial_size=
6  NEO4J_dbms_memory_heap_max_size=
```

.env.example de configuración

Evidencia de Carga de Datos

Información de la BD



Índices creados

The screenshot shows the 'SHOW INDEXES' command results in the Neo4j Desktop interface. The results are displayed in a table with columns: id, name, state, population, type, entityType, labelsOrTypes, properties, indexProvider, owningCons, lastRead, and readCount.

id	name	state	population	type	entityType	labelsOrTypes	properties	indexProvider	owningCons	lastRead	readCount
1	"index_3 43aff4e"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token-1 lookup-1.0"	null	null	0
2	"index_f 7700477"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	null	null	"token-1 lookup-1.0"	null	null	0
3	"movie_title"	"ONLINE"	100.0	"RANGE"	"NODE"	["Movie"]	["title"]	"range-1.0"	"movie_title"	2026-05-20T21:14:51:40.849000000Z	120
5	"person_name"	"ONLINE"	100.0	"RANGE"	"NODE"	["Person"]	["name"]	"range-1.0"	"person_name"	2026-05-20T21:14:51:40.853000000Z	497

Started streaming 4 records after 2 ms and completed after 3 ms.

Consultas iniciales de verificación

The screenshot shows the 'SHOW CONSTRAINTS' command results in the Neo4j Desktop interface. The results are displayed in a table with columns: id, name, type, entityType, labelsOrTypes, properties, ownedIndex, and propertyType.

id	name	type	entityType	labelsOrTypes	properties	ownedIndex	propertyType
4	"movie_title"	"UNIQUENESS"	"NODE"	["Movie"]	["title"]	"movie_title"	null
6	"person_name"	"UNIQUENESS"	"NODE"	["Person"]	["name"]	"person_name"	null

Started streaming 2 records after 17 ms and completed after 19 ms.

The screenshot shows the results of the Cypher query `MATCH (n) RETURN labels(n) AS etiquetas, count(n) AS cantidad ORDER BY cantidad DESC;` in the Neo4j Desktop interface. The results are displayed in a table with columns: etiquetas and cantidad.

etiquetas	cantidad
1 ["Person"]	133
2 ["Movie"]	38

```
neo4j$ MATCH ()-[r]->() RETURN type(r) AS relacion, count(r) AS cantidad ORDER BY cantidad DESC;
```

relacion	cantidad
1 "ACTED_IN"	172
2 "DIRECTED"	44
3 "PRODUCED"	15
4 "WROTE"	10
5 "REVIEWED"	9
6 "FOLLOWS"	3

Consultas de Q1-Q18

Q1 (Media)

Propósito	Listar todas las películas en orden cronológico por fecha y ascendente por nombre.																																																			
Patrón de grafo utilizado	Se utilizó un patrón de nodo simple (m:Movie), que recupera todas las instancias de la etiqueta Movie sin traversal de relaciones.																																																			
Resultado	Las 16 películas entre (1990-1999) con título, año ascendente y lema <table><thead><tr><th>Título</th><th>Año</th><th>Lema</th></tr></thead><tbody><tr><td>1 "Joe Versus the Volcano"</td><td>1990</td><td>"A story of love, lava and burning desire."</td></tr><tr><td>2 "A Few Good Men"</td><td>1992</td><td>"In the heart of the nation's capital, in a courthouse of the U.S. gov ernment, one man will stop at nothing to keep his honor, and one will stop at nothing to find the truth."</td></tr><tr><td>3 "A League of Their Own"</td><td>1992</td><td>"Once in a lifetime you get a chance to do something different."</td></tr><tr><td>4 "Hoffa"</td><td>1992</td><td>"He didn't want law. He wanted justice."</td></tr><tr><td>5 "Unforgiven"</td><td>1992</td><td>"It's a hell of a thing, killing a man"</td></tr><tr><td>6 "Sleepless in Seattle"</td><td>1993</td><td>"What if someone you never met, someone you never saw, someone you nev er knew was the only someone for you?"</td></tr><tr><td>7 "Apollo 13"</td><td>1995</td><td>"Houston, we have a problem."</td></tr><tr><td>8 "Johnny Mnemonic"</td><td>1995</td><td>"The hottest data on earth. In the coolest head in town"</td></tr><tr><td>9 "That Thing You Do"</td><td>1996</td><td>"In every life there comes a time when that thing you dream becomes th at thing you do"</td></tr><tr><td>10 "The Birdcage"</td><td>1996</td><td>"Come as you are"</td></tr><tr><td>11 "Twister"</td><td>1996</td><td>"Don't Breathe. Don't Look Back."</td></tr><tr><td>12 "As Good as It Gets"</td><td>1997</td><td>"A comedy from the heart that goes for the throat."</td></tr><tr><td>13 "The Devil's Advocate"</td><td>1997</td><td>"Evil has its winning ways"</td></tr><tr><td>14 "What Dreams May Come"</td><td>1998</td><td>"After life there is more. The end is just the beginning."</td></tr><tr><td>15 "When Harry Met Sally"</td><td>1998</td><td>"Can two friends sleep together and still love each other in the morni ng?"</td></tr><tr><td>16 "You've Got Mail"</td><td>1998</td><td>"At odds in life... in love on-line."</td></tr></tbody></table>	Título	Año	Lema	1 "Joe Versus the Volcano"	1990	"A story of love, lava and burning desire."	2 "A Few Good Men"	1992	"In the heart of the nation's capital, in a courthouse of the U.S. gov ernment, one man will stop at nothing to keep his honor, and one will stop at nothing to find the truth."	3 "A League of Their Own"	1992	"Once in a lifetime you get a chance to do something different."	4 "Hoffa"	1992	"He didn't want law. He wanted justice."	5 "Unforgiven"	1992	"It's a hell of a thing, killing a man"	6 "Sleepless in Seattle"	1993	"What if someone you never met, someone you never saw, someone you nev er knew was the only someone for you?"	7 "Apollo 13"	1995	"Houston, we have a problem."	8 "Johnny Mnemonic"	1995	"The hottest data on earth. In the coolest head in town"	9 "That Thing You Do"	1996	"In every life there comes a time when that thing you dream becomes th at thing you do"	10 "The Birdcage"	1996	"Come as you are"	11 "Twister"	1996	"Don't Breathe. Don't Look Back."	12 "As Good as It Gets"	1997	"A comedy from the heart that goes for the throat."	13 "The Devil's Advocate"	1997	"Evil has its winning ways"	14 "What Dreams May Come"	1998	"After life there is more. The end is just the beginning."	15 "When Harry Met Sally"	1998	"Can two friends sleep together and still love each other in the morni ng?"	16 "You've Got Mail"	1998	"At odds in life... in love on-line."
Título	Año	Lema																																																		
1 "Joe Versus the Volcano"	1990	"A story of love, lava and burning desire."																																																		
2 "A Few Good Men"	1992	"In the heart of the nation's capital, in a courthouse of the U.S. gov ernment, one man will stop at nothing to keep his honor, and one will stop at nothing to find the truth."																																																		
3 "A League of Their Own"	1992	"Once in a lifetime you get a chance to do something different."																																																		
4 "Hoffa"	1992	"He didn't want law. He wanted justice."																																																		
5 "Unforgiven"	1992	"It's a hell of a thing, killing a man"																																																		
6 "Sleepless in Seattle"	1993	"What if someone you never met, someone you never saw, someone you nev er knew was the only someone for you?"																																																		
7 "Apollo 13"	1995	"Houston, we have a problem."																																																		
8 "Johnny Mnemonic"	1995	"The hottest data on earth. In the coolest head in town"																																																		
9 "That Thing You Do"	1996	"In every life there comes a time when that thing you dream becomes th at thing you do"																																																		
10 "The Birdcage"	1996	"Come as you are"																																																		
11 "Twister"	1996	"Don't Breathe. Don't Look Back."																																																		
12 "As Good as It Gets"	1997	"A comedy from the heart that goes for the throat."																																																		
13 "The Devil's Advocate"	1997	"Evil has its winning ways"																																																		
14 "What Dreams May Come"	1998	"After life there is more. The end is just the beginning."																																																		
15 "When Harry Met Sally"	1998	"Can two friends sleep together and still love each other in the morni ng?"																																																		
16 "You've Got Mail"	1998	"At odds in life... in love on-line."																																																		
Utilidad en un sistema real	Esta consulta resulta útil en sistemas de catálogo cinematográfico o plataformas de streaming, donde se requiere recuperar y presentar películas de una década específica. El filtrado por rango de años permite segmentar el contenido histórico sin cargar datos innecesarios, mientras que el ordenamiento garantiza una presentación cronológica y alfabética consistente para el usuario final.																																																			

Q2 (Media)

Propósito	Listar a los actores de una película específica (matrix) con su respectivo rol/ personaje dentro de la película.																		
Patrón de grafo utilizado	Se usa el nodo (m:Movie {title: "The Matrix"}) como Nodo ancla, es decir, Movie es el nodo inicial de filtrado y bajo este usamos una propiedad para fijar un punto (especificar que es para matrix). En base a ello la consulta es más eficiente porque no debe recorrer películas no necesarias. Seguidamente usa la relación [r: ACTED_IN] como dirección entrante a película y (p:Person) como destino.																		
Resultado	<pre>neo4j\$ MATCH (m:Movie {title: "The Matrix"})<-[r:ACTED_IN]-(p:Person) RETURN p.name AS Persona, p.roles AS Roles</pre> TableRAW <table><thead><tr><th></th><th>Persona</th><th>Roles</th></tr></thead><tbody><tr><td>1</td><td>"Emil Eifrem"</td><td>["Emil"]</td></tr><tr><td>2</td><td>"Hugo Weaving"</td><td>["Agent Smith"]</td></tr><tr><td>3</td><td>"Laurence Fishburne"</td><td>["Morpheus"]</td></tr><tr><td>4</td><td>"Carrie-Anne Moss"</td><td>["Trinity"]</td></tr><tr><td>5</td><td>"Keanu Reeves"</td><td>["Neo"]</td></tr></tbody></table>		Persona	Roles	1	"Emil Eifrem"	["Emil"]	2	"Hugo Weaving"	["Agent Smith"]	3	"Laurence Fishburne"	["Morpheus"]	4	"Carrie-Anne Moss"	["Trinity"]	5	"Keanu Reeves"	["Neo"]
	Persona	Roles																	
1	"Emil Eifrem"	["Emil"]																	
2	"Hugo Weaving"	["Agent Smith"]																	
3	"Laurence Fishburne"	["Morpheus"]																	
4	"Carrie-Anne Moss"	["Trinity"]																	
5	"Keanu Reeves"	["Neo"]																	
Utilidad en un sistema real	Este patrón es útil en sistemas donde se necesita consultar información a partir de un elemento conocido, como en una plataforma de streaming que al seleccionar una película muestra su elenco completo. El nodo ancla evita recorrer todo el grafo, haciendo la consulta eficiente incluso en bases de datos con millones de nodos.																		

Q3 (Media)

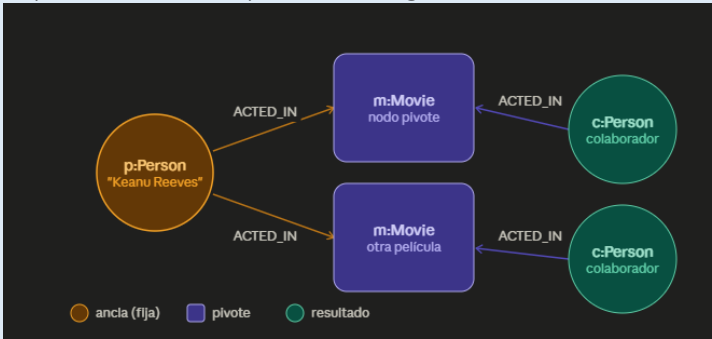
Propósito	Listar todas las películas junto con sus directores, agrupándolas en listas si tienen más de un director.
Patrón de grafo utilizado	<code>(p:Person)-[:DIRECTED]->(m:Movie)</code> Buscamos un nodo que tenga la etiqueta <i>Person</i> apuntando mediante una relación <i>DIRECTED</i> (es decir, dirigió a) hacia un nodo que contenga la etiqueta <i>Movie</i> .

Resultado	<pre>neo4j\$ MATCH (p:Person)-[:DIRECTED]->(m:Movie) RETURN m.title AS Pelicula, collect(p.name) AS Directores ORDER BY Pelicula;</pre> <table><tr><th>Pelicula</th><th>Directores</th></tr><tr><td>1 "A Few Good Men"</td><td>["Rob Reiner"]</td></tr><tr><td>2 "A League of Their Own"</td><td>["Penny Marshall"]</td></tr><tr><td>3 "Apollo 13"</td><td>["Ron Howard"]</td></tr><tr><td>4 "As Good as It Gets"</td><td>["James L. Brooks"]</td></tr><tr><td>5 "Bicentennial Man"</td><td>["Chris Columbus"]</td></tr><tr><td>6 "Cast Away"</td><td>["Robert Zemeckis"]</td></tr><tr><td>7 "Charlie Wilson's War"</td><td>["Mike Nichols"]</td></tr><tr><td>8 "Cloud Atlas"</td><td>["Lana Wachowski", "Lilly Wachowski", "Tom Tykwer"]</td></tr><tr><td>9 "Frost/Nixon"</td><td>["Ron Howard"]</td></tr><tr><td>10 "Hoffa"</td><td>["Danny DeVito"]</td></tr></table> Started streaming 38 records	Pelicula	Directores	1 "A Few Good Men"	["Rob Reiner"]	2 "A League of Their Own"	["Penny Marshall"]	3 "Apollo 13"	["Ron Howard"]	4 "As Good as It Gets"	["James L. Brooks"]	5 "Bicentennial Man"	["Chris Columbus"]	6 "Cast Away"	["Robert Zemeckis"]	7 "Charlie Wilson's War"	["Mike Nichols"]	8 "Cloud Atlas"	["Lana Wachowski", "Lilly Wachowski", "Tom Tykwer"]	9 "Frost/Nixon"	["Ron Howard"]	10 "Hoffa"	["Danny DeVito"]
Pelicula	Directores																						
1 "A Few Good Men"	["Rob Reiner"]																						
2 "A League of Their Own"	["Penny Marshall"]																						
3 "Apollo 13"	["Ron Howard"]																						
4 "As Good as It Gets"	["James L. Brooks"]																						
5 "Bicentennial Man"	["Chris Columbus"]																						
6 "Cast Away"	["Robert Zemeckis"]																						
7 "Charlie Wilson's War"	["Mike Nichols"]																						
8 "Cloud Atlas"	["Lana Wachowski", "Lilly Wachowski", "Tom Tykwer"]																						
9 "Frost/Nixon"	["Ron Howard"]																						
10 "Hoffa"	["Danny DeVito"]																						
Utilidad en un sistema real	Esta es la consulta que se utilizaría en plataformas de streaming para desplegar la información detallada de la película, sin tener que estar duplicando el título de la película varias veces.																						

Q4 (Media)

Propósito	Listar a las personas nacidas después de 1965 que hayan actuado en al menos una película, mostrando información relevante.																																	
Patrón de grafo utilizado	(p:Person)-[:ACTED_IN]->(m:Movie) Se busca cualquier nodo Person que esté asociado por la relación ACTED_IN a un nodo Movie.																																	
Resultado	<pre>neo4j\$ MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE p.born > 1965 RETURN p.name AS Nombre, p.born AS AñoNacimiento, count(m) AS CantidadPeliculas</pre> Table RAW <table><thead><tr><th>Nombre</th><th>AñoNacimiento</th><th>CantidadPeliculas</th></tr></thead><tbody><tr><td>1 "Kiefer Sutherland"</td><td>1966</td><td>2</td></tr><tr><td>2 "John Cusack"</td><td>1966</td><td>1</td></tr><tr><td>3 "Halle Berry"</td><td>1966</td><td>1</td></tr><tr><td>4 "Matthew Fox"</td><td>1966</td><td>1</td></tr><tr><td>5 "Carrie-Anne Moss"</td><td>1967</td><td>3</td></tr><tr><td>6 "James Marshall"</td><td>1967</td><td>1</td></tr><tr><td>7 "Steve Zahn"</td><td>1967</td><td>2</td></tr><tr><td>8 "Ben Miles"</td><td>1967</td><td>3</td></tr><tr><td>9 "Philip Seymour Hoffman"</td><td>1967</td><td>2</td></tr><tr><td>10 "Julia Roberts"</td><td>1967</td><td>1</td></tr></tbody></table> Started streaming	Nombre	AñoNacimiento	CantidadPeliculas	1 "Kiefer Sutherland"	1966	2	2 "John Cusack"	1966	1	3 "Halle Berry"	1966	1	4 "Matthew Fox"	1966	1	5 "Carrie-Anne Moss"	1967	3	6 "James Marshall"	1967	1	7 "Steve Zahn"	1967	2	8 "Ben Miles"	1967	3	9 "Philip Seymour Hoffman"	1967	2	10 "Julia Roberts"	1967	1
Nombre	AñoNacimiento	CantidadPeliculas																																
1 "Kiefer Sutherland"	1966	2																																
2 "John Cusack"	1966	1																																
3 "Halle Berry"	1966	1																																
4 "Matthew Fox"	1966	1																																
5 "Carrie-Anne Moss"	1967	3																																
6 "James Marshall"	1967	1																																
7 "Steve Zahn"	1967	2																																
8 "Ben Miles"	1967	3																																
9 "Philip Seymour Hoffman"	1967	2																																
10 "Julia Roberts"	1967	1																																
Utilidad en un sistema real	Esto puede ayudar a filtrar a las personas que se adecúan a características específicas. Por ejemplo para una agencia de castings que esté buscando a una persona que aún no sea adulto mayor.																																	

Q5 (Media-Alta)

Propósito	Conocer el reparto de películas asociadas a un actor.																														
Patrón de grafo utilizado	El patrón usado se puede ver algo así:  Donde el nodo persona con el filtro inicial de que esa persona sea “Keanu”, es la ancla que nos ahorra consultas innecesarias. Por medio de la relación Acted_In encontramos las películas comunes entre Keanu y X personas en la BD.																														
Resultado	En este caso a la consulta inicial le añadimos una “agregación” de Cypher llamada “collect()” la cual es similar a in Group By en SQL. Sirve para que no salgan actores repetidos con distintas películas, sino que mejor muestra las películas como listas por actor. <table border="1" data-bbox="597 989 1218 1560"> <thead> <tr> <th>Colaborador</th><th>Películas</th></tr> </thead> <tbody> <tr><td>1 "Diane Keaton"</td><td>["Something's Gotta Give"]</td></tr> <tr><td>2 "Jack Nicholson"</td><td>["Something's Gotta Give"]</td></tr> <tr><td>3 "Dina Meyer"</td><td>["Johnny Mnemonic"]</td></tr> <tr><td>4 "Ice-T"</td><td>["Johnny Mnemonic"]</td></tr> <tr><td>5 "Takeshi Kitano"</td><td>["Johnny Mnemonic"]</td></tr> <tr><td>6 "Brooke Langton"</td><td>["The Replacements"]</td></tr> <tr><td>7 "Orlando Jones"</td><td>["The Replacements"]</td></tr> <tr><td>8 "Gene Hackman"</td><td>["The Replacements"]</td></tr> <tr><td>9 "Al Pacino"</td><td>["The Devil's Advocate"]</td></tr> <tr><td>10 "Charlize Theron"</td><td>["The Devil's Advocate"]</td></tr> <tr><td>11 "Hugo Weaving"</td><td>["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]</td></tr> <tr><td>12 "Laurence Fishburne"</td><td>["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]</td></tr> <tr><td>13 "Carrie-Anne Moss"</td><td>["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]</td></tr> <tr><td>14 "Emil Eifrem"</td><td>["The Matrix"]</td></tr> </tbody> </table>	Colaborador	Películas	1 "Diane Keaton"	["Something's Gotta Give"]	2 "Jack Nicholson"	["Something's Gotta Give"]	3 "Dina Meyer"	["Johnny Mnemonic"]	4 "Ice-T"	["Johnny Mnemonic"]	5 "Takeshi Kitano"	["Johnny Mnemonic"]	6 "Brooke Langton"	["The Replacements"]	7 "Orlando Jones"	["The Replacements"]	8 "Gene Hackman"	["The Replacements"]	9 "Al Pacino"	["The Devil's Advocate"]	10 "Charlize Theron"	["The Devil's Advocate"]	11 "Hugo Weaving"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]	12 "Laurence Fishburne"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]	13 "Carrie-Anne Moss"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]	14 "Emil Eifrem"	["The Matrix"]
Colaborador	Películas																														
1 "Diane Keaton"	["Something's Gotta Give"]																														
2 "Jack Nicholson"	["Something's Gotta Give"]																														
3 "Dina Meyer"	["Johnny Mnemonic"]																														
4 "Ice-T"	["Johnny Mnemonic"]																														
5 "Takeshi Kitano"	["Johnny Mnemonic"]																														
6 "Brooke Langton"	["The Replacements"]																														
7 "Orlando Jones"	["The Replacements"]																														
8 "Gene Hackman"	["The Replacements"]																														
9 "Al Pacino"	["The Devil's Advocate"]																														
10 "Charlize Theron"	["The Devil's Advocate"]																														
11 "Hugo Weaving"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]																														
12 "Laurence Fishburne"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]																														
13 "Carrie-Anne Moss"	["The Matrix Revolutions", "The Matrix Reloaded", "The Matrix"]																														
14 "Emil Eifrem"	["The Matrix"]																														
Utilidad en un sistema real	Este patrón es útil en sistemas donde se necesita consultar información a partir de un elemento conocido, como en una plataforma de streaming, al seleccionar un actor podremos ver a los actores con los cuales ha participado en proyectos distintos. Puede servir como método de recomendación en base a actores con los que comparte mucho o tienen carreras similares.																														

Q6 (Media-Alta)

Propósito	Encuentrar personas que hayan participado en películas con al menos dos tipos diferentes de relación entre ACTED_IN, DIRECTED, PRODUCED y WROTE																																	
Patrón de grafo utilizado	(p:Person)-[r:ACTED_IN DIRECTED PRODUCED WROTE]->(m:Movie) El operador en la relación permite buscar multiples tipos de conexión al mismo tiempo. Se le asigna la variable <i>r</i> a la relación para poder saber de qué tipo fue. Se usa WITH para procesar los datos antes de mostrarlos. Usamos type(r) para obtener el nombre de la relación ("ACTED_IN", "DIRECTED", etc.)																																	
Resultado	<pre>neo4j\$ MATCH (p:Person)-[r:ACTED_IN DIRECTED PRODUCED WROTE]->(m:Movie) WITH p, collect(DISTINCT type(r)) AS TiposParticipacion, count(D</pre> Table RAW <table><tr><th>Persona</th><th>TiposParticipacion</th><th>CantidadPeliculas</th></tr><tr><td>1 "Tom Hanks"</td><td>["ACTED_IN", "DIRECTED"]</td><td>12</td></tr><tr><td>2 "Lana Wachowski"</td><td>["DIRECTED", "WROTE", "PRODUCED"]</td><td>7</td></tr><tr><td>3 "Lilly Wachowski"</td><td>["DIRECTED", "WROTE", "PRODUCED"]</td><td>7</td></tr><tr><td>4 "Rob Reiner"</td><td>["DIRECTED", "PRODUCED"]</td><td>3</td></tr><tr><td>5 "James Marshall"</td><td>["ACTED_IN", "DIRECTED"]</td><td>3</td></tr><tr><td>6 "Nora Ephron"</td><td>["DIRECTED", "PRODUCED", "WROTE"]</td><td>3</td></tr><tr><td>7 "Werner Herzog"</td><td>["ACTED_IN", "DIRECTED"]</td><td>2</td></tr><tr><td>8 "Danny DeVito"</td><td>["ACTED_IN", "DIRECTED"]</td><td>2</td></tr><tr><td>9 "Aaron Sorkin"</td><td>["WROTE", "ACTED_IN"]</td><td>1</td></tr><tr><td>10 "Cameron Crowe"</td><td>["WROTE", "PRODUCED", "DIRECTED"]</td><td>1</td></tr></table>	Persona	TiposParticipacion	CantidadPeliculas	1 "Tom Hanks"	["ACTED_IN", "DIRECTED"]	12	2 "Lana Wachowski"	["DIRECTED", "WROTE", "PRODUCED"]	7	3 "Lilly Wachowski"	["DIRECTED", "WROTE", "PRODUCED"]	7	4 "Rob Reiner"	["DIRECTED", "PRODUCED"]	3	5 "James Marshall"	["ACTED_IN", "DIRECTED"]	3	6 "Nora Ephron"	["DIRECTED", "PRODUCED", "WROTE"]	3	7 "Werner Herzog"	["ACTED_IN", "DIRECTED"]	2	8 "Danny DeVito"	["ACTED_IN", "DIRECTED"]	2	9 "Aaron Sorkin"	["WROTE", "ACTED_IN"]	1	10 "Cameron Crowe"	["WROTE", "PRODUCED", "DIRECTED"]	1
Persona	TiposParticipacion	CantidadPeliculas																																
1 "Tom Hanks"	["ACTED_IN", "DIRECTED"]	12																																
2 "Lana Wachowski"	["DIRECTED", "WROTE", "PRODUCED"]	7																																
3 "Lilly Wachowski"	["DIRECTED", "WROTE", "PRODUCED"]	7																																
4 "Rob Reiner"	["DIRECTED", "PRODUCED"]	3																																
5 "James Marshall"	["ACTED_IN", "DIRECTED"]	3																																
6 "Nora Ephron"	["DIRECTED", "PRODUCED", "WROTE"]	3																																
7 "Werner Herzog"	["ACTED_IN", "DIRECTED"]	2																																
8 "Danny DeVito"	["ACTED_IN", "DIRECTED"]	2																																
9 "Aaron Sorkin"	["WROTE", "ACTED_IN"]	1																																
10 "Cameron Crowe"	["WROTE", "PRODUCED", "DIRECTED"]	1																																
Utilidad en un sistema real	Sirve para encontrar personas con un rango de especialidad más amplio. Como actores que también han dirigido. Esto es particularmente útil para una agencia que busca versatilidad en trabajadores.																																	

Q7 (Media-Alta)

Propósito	Listar las películas que tengan un reparto de N tamaño.																
Patrón de grafo utilizado	(m:Movie)-[:ACTED_IN]-(p:Person) Movie es el nodo ancla, en base a el exploramos las distintas relaciones que tiene con las Personas para determinar el tamaño del reparto.																
Resultado	<table><thead><tr><th>Pelicula</th><th>Año</th><th>NúmeroDeActores</th><th>Actores</th></tr></thead><tbody><tr><td>1 "A Few Good Men"</td><td>1992</td><td>12</td><td>["Christopher Guest", "J.T. Walsh", "Kevin Pollak", "Aaron Sorkin", "James Marshall", "Cuba Gooding Jr.", "Noah Wyle", "Kiefer Sutherland", "Kevin Bacon", "Demi Moore", "Jack Nicholson", "Tom Cruise"]</td></tr><tr><td>2 "Top Gun"</td><td>1986</td><td>6</td><td>["Val Kilmer", "Kelly McGillis", "Tom Cruise", "Anthony Edwards", "Meg Ryan", "Tom Skerritt"]</td></tr><tr><td>3 "Jerry Maguire"</td><td>2000</td><td>9</td><td>["Jerry O'Connell", "Cuba Gooding Jr.", "Jonathan Lipnicki", "Regina King", "Bonnie Hunt", "Renee Zellweger", "Tom Cruise", "Jay Mohr", "Kelly Preston"]</td></tr></tbody></table>	Pelicula	Año	NúmeroDeActores	Actores	1 "A Few Good Men"	1992	12	["Christopher Guest", "J.T. Walsh", "Kevin Pollak", "Aaron Sorkin", "James Marshall", "Cuba Gooding Jr.", "Noah Wyle", "Kiefer Sutherland", "Kevin Bacon", "Demi Moore", "Jack Nicholson", "Tom Cruise"]	2 "Top Gun"	1986	6	["Val Kilmer", "Kelly McGillis", "Tom Cruise", "Anthony Edwards", "Meg Ryan", "Tom Skerritt"]	3 "Jerry Maguire"	2000	9	["Jerry O'Connell", "Cuba Gooding Jr.", "Jonathan Lipnicki", "Regina King", "Bonnie Hunt", "Renee Zellweger", "Tom Cruise", "Jay Mohr", "Kelly Preston"]
Pelicula	Año	NúmeroDeActores	Actores														
1 "A Few Good Men"	1992	12	["Christopher Guest", "J.T. Walsh", "Kevin Pollak", "Aaron Sorkin", "James Marshall", "Cuba Gooding Jr.", "Noah Wyle", "Kiefer Sutherland", "Kevin Bacon", "Demi Moore", "Jack Nicholson", "Tom Cruise"]														
2 "Top Gun"	1986	6	["Val Kilmer", "Kelly McGillis", "Tom Cruise", "Anthony Edwards", "Meg Ryan", "Tom Skerritt"]														
3 "Jerry Maguire"	2000	9	["Jerry O'Connell", "Cuba Gooding Jr.", "Jonathan Lipnicki", "Regina King", "Bonnie Hunt", "Renee Zellweger", "Tom Cruise", "Jay Mohr", "Kelly Preston"]														

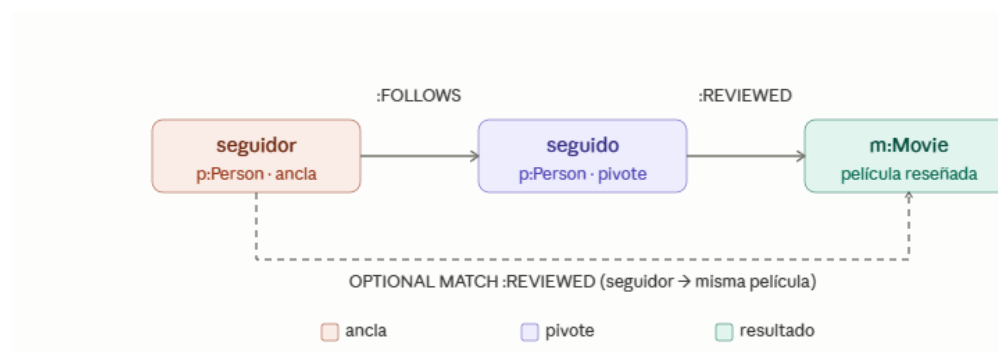
	4 "Stand By Me" 1986 7 ["River Phoenix", "Kiefer Sutherland", "Marshall Bell", "Corey Feldman", "Jerry O'Connell", "John Cusack", "Wil Wheaton"] 5 "You've Got Mail" 1998 6 ["Meg Ryan", "Tom Hanks", "Steve Zahn", "Parker Posey", "Greg Kinnear", "Dave Chappelle"] 6 "Sleepless in Seattle" 1993 6 ["Victor Garber", "Rita Wilson", "Bill Pullman", "Rosie O'Donnell", "Meg Ryan", "Tom Hanks"] 7 "Speed Racer" 2008 7 ["John Goodman", "Matthew Fox", "Emile Hirsch", "Susan Sarandon", "Rainn", "Ben Miles", "Christina Ricci"] 8 "The Green Mile"  1999 8 ["Sam Rockwell", "James Cromwell", "David Morse", "Gary Sinise", "Patricia Clarkson", "Tom Hanks", "Bonnie Hunt", "Michael Clarke Duncan"] 9 "A League of Their Own" 1992 6 ["Geena Davis", "Bill Paxton", "Madonna", "Lori Petty", "Tom Hanks", "Rosie O'Donnell"]
Utilidad en un sistema real	Este patrón es útil en sistemas donde se necesita consultar el reparto de alguna película, ya sea para enumerar actores, estudiar el tamaño que tuvo este cast con respecto a la Budget de la película o estudiar la magnitud de la producción necesaria para futuras iteraciones de este formato.

Q8 (Media-Alta)

Propósito	Usar las relaciones :FOLLOWS y :REVIEWED, y listar las películas reseñadas por las personas seguidas																																																							
Patrón de grafo utilizado	Se navega el grafo desde un usuario <i>seguidor</i> hacia otro <i>seguido</i> a través de :FOLLOWS, y luego de ese seguido hacia una película <i>m</i> mediante :REVIEWED. Luego se usa OPTIONAL MATCH para buscar si el <i>seguidor</i> también hizo una relación :REVIEWED hacia la <i>misma</i> película <i>m</i> . El OPTIONAL MATCH es como un LEFT JOIN en SQL, es decir, busca la conexión, pero si no la encuentra, no elimina el resultado, simplemente devuelve un valor.																																																							
Resultado	<pre>neo4j> MATCH (seguidor:Person)-[:FOLLOWS]->(seguido:Person)-[r2:REVIEWED]->(m:Movie) OPTIONAL MATCH (seguidor)-[r1:REVIEWED]->(m) RETURN seguidor.name</pre> Table RAW <table><thead><tr><th>QuienSigue</th><th>AQuienSigue</th><th>PeliculaResenada</th><th>Calificacion</th><th>SeguidorTambienReseno</th></tr></thead><tbody><tr><td>4 "Angela Scope"</td><td>"Jessica Thompson"</td><td>"The Replacements"</td><td>65</td><td>true</td></tr><tr><td>5 "Angela Scope"</td><td>"Jessica Thompson"</td><td>"The Da Vinci Code"</td><td>68</td><td>false</td></tr><tr><td>6 "Angela Scope"</td><td>"Jessica Thompson"</td><td>"The Birdcage"</td><td>45</td><td>false</td></tr><tr><td>7 "James Thompson"</td><td>"Jessica Thompson"</td><td>"Unforgiven"</td><td>85</td><td>false</td></tr><tr><td>8 "James Thompson"</td><td>"Jessica Thompson"</td><td>"Jerry Maguire"</td><td>92</td><td>false</td></tr><tr><td>9 "James Thompson"</td><td>"Jessica Thompson"</td><td>"Cloud Atlas"</td><td>95</td><td>false</td></tr><tr><td>10 "James Thompson"</td><td>"Jessica Thompson"</td><td>"The Replacements"</td><td>65</td><td>true</td></tr><tr><td>11 "James Thompson"</td><td>"Jessica Thompson"</td><td>"The Da Vinci Code"</td><td>68</td><td>true</td></tr><tr><td>12 "James Thompson"</td><td>"Jessica Thompson"</td><td>"The Birdcage"</td><td>45</td><td>false</td></tr><tr><td>13 "Paul Blythe"</td><td>"Angela Scope"</td><td>"The Replacements"</td><td>62</td><td>false</td></tr></tbody></table>	QuienSigue	AQuienSigue	PeliculaResenada	Calificacion	SeguidorTambienReseno	4 "Angela Scope"	"Jessica Thompson"	"The Replacements"	65	true	5 "Angela Scope"	"Jessica Thompson"	"The Da Vinci Code"	68	false	6 "Angela Scope"	"Jessica Thompson"	"The Birdcage"	45	false	7 "James Thompson"	"Jessica Thompson"	"Unforgiven"	85	false	8 "James Thompson"	"Jessica Thompson"	"Jerry Maguire"	92	false	9 "James Thompson"	"Jessica Thompson"	"Cloud Atlas"	95	false	10 "James Thompson"	"Jessica Thompson"	"The Replacements"	65	true	11 "James Thompson"	"Jessica Thompson"	"The Da Vinci Code"	68	true	12 "James Thompson"	"Jessica Thompson"	"The Birdcage"	45	false	13 "Paul Blythe"	"Angela Scope"	"The Replacements"	62	false
QuienSigue	AQuienSigue	PeliculaResenada	Calificacion	SeguidorTambienReseno																																																				
4 "Angela Scope"	"Jessica Thompson"	"The Replacements"	65	true																																																				
5 "Angela Scope"	"Jessica Thompson"	"The Da Vinci Code"	68	false																																																				
6 "Angela Scope"	"Jessica Thompson"	"The Birdcage"	45	false																																																				
7 "James Thompson"	"Jessica Thompson"	"Unforgiven"	85	false																																																				
8 "James Thompson"	"Jessica Thompson"	"Jerry Maguire"	92	false																																																				
9 "James Thompson"	"Jessica Thompson"	"Cloud Atlas"	95	false																																																				
10 "James Thompson"	"Jessica Thompson"	"The Replacements"	65	true																																																				
11 "James Thompson"	"Jessica Thompson"	"The Da Vinci Code"	68	true																																																				
12 "James Thompson"	"Jessica Thompson"	"The Birdcage"	45	false																																																				
13 "Paul Blythe"	"Angela Scope"	"The Replacements"	62	false																																																				
Utilidad en un sistema real	Sirve para construir los feeds de las redes sociales, en donde se puede mostrar la interacción de otras personas que sigo o me siguen con el contenido que yo interactúo.																																																							

Análisis: el patrón utiliza dos niveles desde el nodo ancla seguidor. Por lo que el recorrido avanza por :FOLLOWS hacia el seguido que funge como pivote, y luego pasa por :REVIEWED hacia la película resultado. El OPTIONAL MATCH funciona similar a un left join, donde intenta extender cada fila buscando si el propio seguidor también tiene una relación :REVIEWED hacia esa misma película m. Si no la encuentra, entonces devuelve un null en la columna.

No4j materializa el resultado del MATCH principal en un conjunto intermedio de filas, intentando resolver el OPTIONAL MATCH independientemente, logrando evitar hacer un producto cartesiano que acabaría en un segundo MATCH completo. Luego, el nodo seguidor actúa como ancla fija gracias al filtro inicial en el nombre, restringiendo el punto de entrada del traversal y evitando recorrer toda la base.



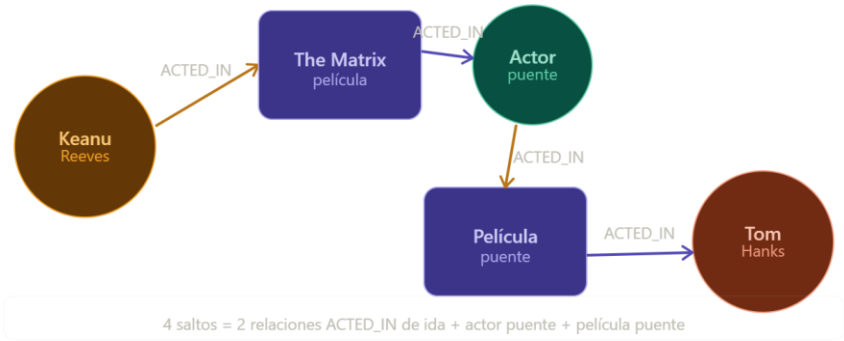
Q9 (Difícil)

Propósito	Determinar cuál es la ruta más corta de consulta en la que podemos ir del nodo Keanu -> Tom Hanks. Esto es útil para optimizar y ahorrar recursos.
Patrón de grafo utilizado	Se utilizó un patrón de traversal sin dirección fija entre dos nodos ancla Person, es decir, establecimos el nodo inicio y el final, sabiendo que se conectan a través de relaciones ACTED_IN de longitud variable. Tal que la ruta completa, incluyendo nodos intermedios de tipo Movie, es capturada en la variable ruta, la cual hace uso de una función de Neo4j llamada "shortestPath()" que implementa un BFS interno para encontrar el camino más corto en el grafo.

Resultado	<table><thead><tr><th>Ruta</th><th>Salto</th></tr></thead><tbody><tr><td>["Keanu Reeves", "The Devil's Advocate", "Charlize Theron", "That Thing You Do", "Tom Hanks"]</td><td>4</td></tr></tbody></table>	Ruta	Salto	["Keanu Reeves", "The Devil's Advocate", "Charlize Theron", "That Thing You Do", "Tom Hanks"]	4
Ruta	Salto				
["Keanu Reeves", "The Devil's Advocate", "Charlize Theron", "That Thing You Do", "Tom Hanks"]	4				
Utilidad en un sistema real	Aplicable en redes sociales o plataformas de entretenimiento para calcular el grado de separación entre personas, similar al concepto de los 'seis grados de separación'. Por ejemplo, en una plataforma como IMDb podría usarse para mostrar cómo dos actores están conectados a través de colaboraciones, o en redes sociales para buscar amigos en cierto rango.				

Análisis: Para encontrar la ruta más corta en un grafo sabiendo en que nodo empezamos y donde queremos terminar (Keanu-Tom), Neo4j nos permite movernos usando las relaciones y nodos intermedios (películas). En este caso la función de shortestPath() es la clave, pues implemente el BFS por anchura para determinar todos los caminos posibles y entre ellos se elige el más corto.

A nivel más técnico lo que sucede en esta consulta es: En el match definimos 2 nodos ancla independientes, el inicio y el destino, en base a esos puntos se aplica la función de shortestPath() y encuentra la ruta sin tomar en cuenta alguna dirección fija, es decir que atraviesa en cualquier sentido de la relación al usar el [:ACTED_IN*], esto se plantea así ya que el grafo alterna entre person->acted_in->movie al no haber una relación directa de person->person . Seguidamente se usa una lista de compresión en el RETURN, que se encarga de clasificar los nodos pares en Movie e impar en Person, esto para contar las relaciones NO los nodos.



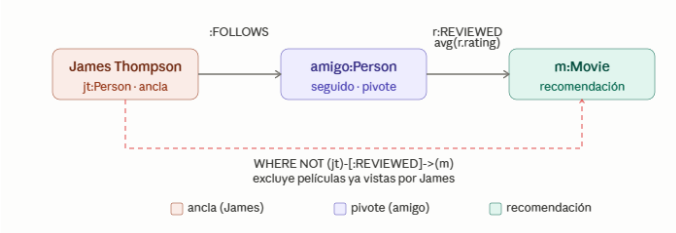
Q10 (Difícil)

Propósito	Generar recomendaciones para James Thompson, por personas a las que él sigue.
Patrón de grafo utilizado	(jt)-[:FOLLOWS]->(amigo)-[r:REVIEWED]->(m) busca todas las películas que han visto las personas a las que James sigue.Luego, la condición WHERE NOT (jt)-[:REVIEWED]->(m) evalúa la topología del grafo para excluir cualquier nodo Movie

	que ya tenga una relación de reseña apuntando directamente desde James. Al retornar las películas, se usa collect(amigo.name) para agrupar en una lista quiénes la recomendaron, y avg(r.rating) para calcular automáticamente el promedio matemático de la calificación dada por esos contactos.															
Resultado	<pre>neo4j\$ MATCH (jt:Person {name: 'James Thompson'})-[:FOLLOWS]->(amigo:Person)-[r:REVIEWED]->(m:Movie) WHERE NOT (jt)-[:REVIEWED]->(m) RETURN</pre> Table RAW <table><tr><th>PeliculaRecomendada</th><th>RecomendadaPor</th><th>CalificacionPromedio</th></tr><tr><td>"Cloud Atlas"</td><td>["Jessica Thompson"]</td><td>95.0</td></tr><tr><td>"Jerry Maguire"</td><td>["Jessica Thompson"]</td><td>92.0</td></tr><tr><td>"Unforgiven"</td><td>["Jessica Thompson"]</td><td>85.0</td></tr><tr><td>"The Birdcage"</td><td>["Jessica Thompson"]</td><td>45.0</td></tr></table>	PeliculaRecomendada	RecomendadaPor	CalificacionPromedio	"Cloud Atlas"	["Jessica Thompson"]	95.0	"Jerry Maguire"	["Jessica Thompson"]	92.0	"Unforgiven"	["Jessica Thompson"]	85.0	"The Birdcage"	["Jessica Thompson"]	45.0
PeliculaRecomendada	RecomendadaPor	CalificacionPromedio														
"Cloud Atlas"	["Jessica Thompson"]	95.0														
"Jerry Maguire"	["Jessica Thompson"]	92.0														
"Unforgiven"	["Jessica Thompson"]	85.0														
"The Birdcage"	["Jessica Thompson"]	45.0														
Utilidad en un sistema real	Es la manera en la que en los feeds generan contenido recomendado que uno no ha visto, pero que las personas a las que sigo sí les gusta, por lo que es posible que me interese a mí también.															

Análisis: para este caso, el patrón extiende el traversal de Q8 hacia un sistema de recomendación. Entonces James Thompson actúa como nodo ancla fijo donde pasa por dos niveles: primero :FOLLOWS para llegar a sus contactos, y luego :REVIEWED para alcanzar películas que esos contactos calificaron. La condición WHERE NOT (jt)-[:REVIEWED]->(m) es un filtro de topología del grafo, lo que significa que descarta cualquier nodo Movie que ya tenga una arista directa de :REVIEWED apuntando desde James, sin requerir subconsultas ni joins adicionales.

El RETURN utiliza collect(amigo.name) para agrupar en una lista los "recomendadores" de cada película, y avg(r.rating) para calcular automáticamente el promedio de calificación dado por esos contactos. Entonces se produce un puntaje de recomendación natural: a mayor cantidad de contactos que reseñaron la película y mayor su calificación promedio, más fuerte es la sugerencia. A nivel de ejecución, Neo4j evalúa el WHERE NOT como una comprobación de existencia de arista en la lista de adyacencia de James, lo que es una operación de tiempo constante para cada película candidata.

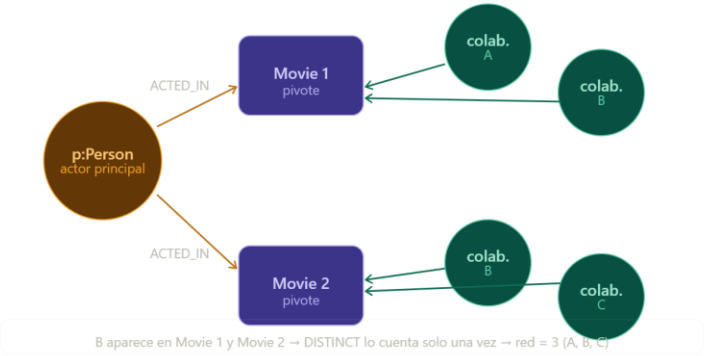


Q11 (Difícil)

Propósito	Calcular con cuántos distintos actores ha trabajado un actor, así establecer un ranking.
-----------	--

Patrón de grafo utilizado	Se utilizó un patrón de traversal doble donde desde cada nodo Person se recorren relaciones salientes ACTED_IN hacia nodos Movie, y desde cada película se recorren relaciones entrantes ACTED_IN para alcanzar otros nodos Person colaboradores. Es el mismo patrón pivote de la Q9 pero aplicado a todos los actores del grafo, sin nodo ancla fijo.																						
Resultado	<table><thead><tr><th>Actor</th><th>RedDeColaboración</th></tr></thead><tbody><tr><td>1 "Tom Hanks"</td><td>34</td></tr><tr><td>2 "Cuba Gooding Jr."</td><td>24</td></tr><tr><td>3 "Tom Cruise"</td><td>23</td></tr><tr><td>4 "Kevin Bacon"</td><td>19</td></tr><tr><td>5 "Meg Ryan"</td><td>18</td></tr><tr><td>6 "Kiefer Sutherland"</td><td>17</td></tr><tr><td>7 "Jack Nicholson"</td><td>17</td></tr><tr><td>8 "Bonnie Hunt"</td><td>15</td></tr><tr><td>9 "Keanu Reeves"</td><td>14</td></tr><tr><td>10 "Jerry O'Connell"</td><td>14</td></tr></tbody></table>	Actor	RedDeColaboración	1 "Tom Hanks"	34	2 "Cuba Gooding Jr."	24	3 "Tom Cruise"	23	4 "Kevin Bacon"	19	5 "Meg Ryan"	18	6 "Kiefer Sutherland"	17	7 "Jack Nicholson"	17	8 "Bonnie Hunt"	15	9 "Keanu Reeves"	14	10 "Jerry O'Connell"	14
Actor	RedDeColaboración																						
1 "Tom Hanks"	34																						
2 "Cuba Gooding Jr."	24																						
3 "Tom Cruise"	23																						
4 "Kevin Bacon"	19																						
5 "Meg Ryan"	18																						
6 "Kiefer Sutherland"	17																						
7 "Jack Nicholson"	17																						
8 "Bonnie Hunt"	15																						
9 "Keanu Reeves"	14																						
10 "Jerry O'Connell"	14																						
Utilidad en un sistema real	Útil en plataformas de casting o agencias de talento para identificar actores altamente conectados dentro de la industria, ya que aquellos con mayor red de colaboración tienen más influencia y visibilidad. También aplicable en análisis de redes sociales para detectar nodos con alto grado de conectividad.																						

Análisis: Para encontrar la red de colaboración lo que hace Neo4j es buscar para cada actor todas sus películas, y de estas buscar los actores que salen de ellas. Lo esencial fue usar el DISTINCT para que no se cuenten 2 veces los actores.



A nivel técnico el patrón usado aquí trae todas las películas por actor y todos los actores de la relación con película, en otras palabras, es un producto cartesiano. El resultado se encargaa de colapsar las filas para mostrar actores únicos con count(DISTINCT c), para después ordenarlo descendentemente y sacar el top 10 de quienes tiene una mayor red.

Q12 (Difícil)

Propósito	Calcular una versión simplificada del número de Bacon para las personas conectadas con Kevin Bacon mediante relaciones :ACTED_IN																						
Patrón de grafo utilizado	Se emplea shortestPath() para encontrar la ruta más corta entre Kevin Bacon y cualquier otro nodo <i>Person</i> . Cuando se pone [:ACTED_IN*] se le está indicando a Neo4j que navegue a través de la relación de actuación sin importar la cantidad de saltos. Como la relación no tiene flecha direccional, busca en ambas vías. Dado, que el grafo de Neo4j es Persona y Película, la ruta siempre hace Persona-> Película-> Persona. Por esto, el tamaño total de la ruta (length(p)) se divide entre 2 para obtener la cantidad real de películas que los separan.																						
Resultado	<pre>neo4j\$ MATCH p = shortestPath((bacon:Person {name: 'Kevin Bacon'})-[:ACTED_IN*]-(otro:Person)) WHERE bacon <> otro</pre> Table RAW <table> <thead> <tr> <th>Persona</th><th>NumeroDeBacon</th></tr> </thead> <tbody> <tr><td>1 "Aaron Sorkin"</td><td>1</td></tr> <tr><td>2 "Bill Paxton"</td><td>1</td></tr> <tr><td>3 "Christopher Guest"</td><td>1</td></tr> <tr><td>4 "Cuba Gooding Jr."</td><td>1</td></tr> <tr><td>5 "Demi Moore"</td><td>1</td></tr> <tr><td>6 "Ed Harris"</td><td>1</td></tr> <tr><td>7 "Frank Langella"</td><td>1</td></tr> <tr><td>8 "Gary Sinise"</td><td>1</td></tr> <tr><td>9 "J.T. Walsh"</td><td>1</td></tr> <tr><td>10 "Jack Nicholson"</td><td>1</td></tr> </tbody> </table>	Persona	NumeroDeBacon	1 "Aaron Sorkin"	1	2 "Bill Paxton"	1	3 "Christopher Guest"	1	4 "Cuba Gooding Jr."	1	5 "Demi Moore"	1	6 "Ed Harris"	1	7 "Frank Langella"	1	8 "Gary Sinise"	1	9 "J.T. Walsh"	1	10 "Jack Nicholson"	1
Persona	NumeroDeBacon																						
1 "Aaron Sorkin"	1																						
2 "Bill Paxton"	1																						
3 "Christopher Guest"	1																						
4 "Cuba Gooding Jr."	1																						
5 "Demi Moore"	1																						
6 "Ed Harris"	1																						
7 "Frank Langella"	1																						
8 "Gary Sinise"	1																						
9 "J.T. Walsh"	1																						
10 "Jack Nicholson"	1																						
Utilidad en un sistema real	Sirve para consultar rutas cortas en sistemas que requieren logística como aplicaciones que usan gps.																						

Análisis: este patrón aplica shortestPath() con Kevin Bacon como nodo ancla fijo y observa la distancia hacia todos los demás nodos Person del grafo. La relación [:ACTED_IN*] sin dirección lo que hace es que alterna el grafo real alterna Person → Movie → Person porque tanto Kevin como el otro actor tienen relaciones salientes ACTED_IN hacia la película compartida. Sin el traversal bidireccional, Neo4j no podría cruzar el nodo Movie pivote y la ruta quedaría incompleta.

La división ($\text{length}(p) / 2$) convierte el número total de aristas de la ruta en el número real de películas que separan a ambas personas. Cada "conexión real" entre dos actores involucra siempre dos aristas: una $\text{Person} \rightarrow \text{Movie}$ y una $\text{Movie} \rightarrow \text{Person}$, por lo que el número de películas es exactamente la mitad de la longitud total de la ruta. A nivel de implementación, `shortestPath()` ejecuta un BFS bidireccional interno que se detiene al encontrar la primera ruta mínima, lo que es significativamente más eficiente que un traversal de profundidad variable con `[:ACTED_IN*]` sin restricción de longitud.

Q13 (Difícil)

Propósito	Definir cuales personas han trabajado como actor y como director.																		
Patrón de grafo utilizado	Se utilizaron dos patrones de transversal independientes sobre el mismo nodo ancla Person: uno siguiendo relaciones salientes ACTED_IN hacia películas en las que actuó, y otro siguiendo relaciones salientes DIRECTED hacia películas que dirigió. El hecho de compartir el nodo p obliga a que ambas condiciones se cumplan en la misma persona.																		
Resultado	<table><thead><tr><th>Persona</th><th>Actuó</th><th>Dirigió</th></tr></thead><tbody><tr><td>1 "Clint Eastwood"</td><td>["Unforgiven"]</td><td>["Unforgiven"]</td></tr><tr><td>2 "Danny DeVito"</td><td>["One Flew Over the Cuckoo's Nest", "Hoffa"]</td><td>["Hoffa"]</td></tr><tr><td>3 "James Marshall"</td><td>["A Few Good Men"]</td><td>["V for Vendetta", "Ninja Assassin"]</td></tr><tr><td>4 "Tom Hanks"</td><td>["A League of Their Own", "The Polar Express", "Charlie Wilson's War", "Cast Away", "Apollo 13", "The Green Mile", "The Da Vinci Code", "Cloud Atlas", "That Thing You Do", "Joe Versus the Volcano", "Sleepless in Seattle", "You've Got Mail"]</td><td>["That Thing You Do"]</td></tr><tr><td>5 "Werner Herzog"</td><td>["What Dreams May Come"]</td><td>["RescueDawn"]</td></tr></tbody></table>	Persona	Actuó	Dirigió	1 "Clint Eastwood"	["Unforgiven"]	["Unforgiven"]	2 "Danny DeVito"	["One Flew Over the Cuckoo's Nest", "Hoffa"]	["Hoffa"]	3 "James Marshall"	["A Few Good Men"]	["V for Vendetta", "Ninja Assassin"]	4 "Tom Hanks"	["A League of Their Own", "The Polar Express", "Charlie Wilson's War", "Cast Away", "Apollo 13", "The Green Mile", "The Da Vinci Code", "Cloud Atlas", "That Thing You Do", "Joe Versus the Volcano", "Sleepless in Seattle", "You've Got Mail"]	["That Thing You Do"]	5 "Werner Herzog"	["What Dreams May Come"]	["RescueDawn"]
Persona	Actuó	Dirigió																	
1 "Clint Eastwood"	["Unforgiven"]	["Unforgiven"]																	
2 "Danny DeVito"	["One Flew Over the Cuckoo's Nest", "Hoffa"]	["Hoffa"]																	
3 "James Marshall"	["A Few Good Men"]	["V for Vendetta", "Ninja Assassin"]																	
4 "Tom Hanks"	["A League of Their Own", "The Polar Express", "Charlie Wilson's War", "Cast Away", "Apollo 13", "The Green Mile", "The Da Vinci Code", "Cloud Atlas", "That Thing You Do", "Joe Versus the Volcano", "Sleepless in Seattle", "You've Got Mail"]	["That Thing You Do"]																	
5 "Werner Herzog"	["What Dreams May Come"]	["RescueDawn"]																	
Utilidad en un sistema real	Útil en plataformas de cine o bases de datos de la industria para identificar personas con doble rol creativo, como actores-directores. Podría alimentar filtros de búsqueda avanzada o secciones editoriales del tipo 'directores que también actúan'.																		

Análisis: La idea de esta consulta es buscar una persona con doble relación sobre la película tal que se usa el mismo nodo `p:Person` como una especie de JOIN implícito, donde solo se mantienen aquellos resultados que cumplan con la clausula del `MATCH` simultáneamente. De esta manera si una persona tiene la relación `ACTED_IN` pero no `DIRECTED`, queda eliminada y se ahorra tener que recorrer esa relación. Esto implica que el nodo `p` debe tener ambas relaciones, lo que internamente provoca un plano cartesiano interno con todos los dirigidos y actuados, por ello se usa el `DISTINCT` dentro de un `collect()`, así se agrupa y a la vez evita duplicados.

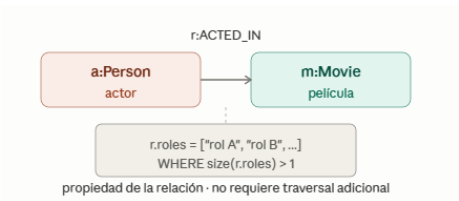
Q14 (Difícil)

Propósito	Encontrar actores que interpretaron más de un rol en una misma película.
------------------	--

Patrón de grafo utilizado	(a:Person)-[r:ACTED_IN]->(m:Movie), conecta a actor con película y capturando la variable de la relación. A diferencia de las consultas anteriores que evalúan conexiones, aquí el peso recae en las <i>propiedades</i> de la relación. La relación ACTED_IN tiene un atributo llamado roles que guarda una lista. Usando size(r.roles) > 1 se filtra únicamente a los actores cuyo arreglo interno de personajes tenga 2 o más elementos.																												
Resultado	<pre>neo4j\$ MATCH (a:Person)-[r:ACTED_IN]->(m:Movie) WHERE size(r.roles) > 1 RETURN a.name AS Actor, m.title AS Pelicula, r.roles AS ListaDeRoles, size(r.roles) AS CantidadDeRoles</pre> Table RAW <table><tr><th>Actor</th><th>Pelicula</th><th>ListaDeRoles</th><th>CantidadDeRoles</th></tr><tr><td>1 "Hugo Weaving"</td><td>"Cloud Atlas"</td><td>["Bill Smoke", "Haskell Moore", "Tadeusz Kesselring", "Nurse Noakes", "Boardman Phillips", "Old Georgie"]</td><td>6</td></tr><tr><td>2 "Tom Hanks"</td><td>"The Polar Express"</td><td>["Hero Boy", "Father", "Conductor", "Hobo", "Scrooge", "Santa Claus"]</td><td>6</td></tr><tr><td>3 "Halle Berry"</td><td>"Cloud Atlas"</td><td>["Luisa Rey", "Jocasta Ayrs", "Ovid", "Meronym"]</td><td>4</td></tr><tr><td>4 "Tom Hanks"</td><td>"Cloud Atlas"</td><td>["Zachry", "Dr. Henry Goose", "Isaac Sachs", "Dermot Hoggins"]</td><td>4</td></tr><tr><td>5 "Jim Broadbent"</td><td>"Cloud Atlas"</td><td>["Vyvyan Ayrs", "Captain Molyneux", "Timothy Cavendish"]</td><td>3</td></tr><tr><td>6 "Meg Ryan"</td><td>"Joe Versus the Volcano"</td><td>["DeDe", "Angelica Graynamore", "Patricia Graynamore"]</td><td>3</td></tr></table>	Actor	Pelicula	ListaDeRoles	CantidadDeRoles	1 "Hugo Weaving"	"Cloud Atlas"	["Bill Smoke", "Haskell Moore", "Tadeusz Kesselring", "Nurse Noakes", "Boardman Phillips", "Old Georgie"]	6	2 "Tom Hanks"	"The Polar Express"	["Hero Boy", "Father", "Conductor", "Hobo", "Scrooge", "Santa Claus"]	6	3 "Halle Berry"	"Cloud Atlas"	["Luisa Rey", "Jocasta Ayrs", "Ovid", "Meronym"]	4	4 "Tom Hanks"	"Cloud Atlas"	["Zachry", "Dr. Henry Goose", "Isaac Sachs", "Dermot Hoggins"]	4	5 "Jim Broadbent"	"Cloud Atlas"	["Vyvyan Ayrs", "Captain Molyneux", "Timothy Cavendish"]	3	6 "Meg Ryan"	"Joe Versus the Volcano"	["DeDe", "Angelica Graynamore", "Patricia Graynamore"]	3
Actor	Pelicula	ListaDeRoles	CantidadDeRoles																										
1 "Hugo Weaving"	"Cloud Atlas"	["Bill Smoke", "Haskell Moore", "Tadeusz Kesselring", "Nurse Noakes", "Boardman Phillips", "Old Georgie"]	6																										
2 "Tom Hanks"	"The Polar Express"	["Hero Boy", "Father", "Conductor", "Hobo", "Scrooge", "Santa Claus"]	6																										
3 "Halle Berry"	"Cloud Atlas"	["Luisa Rey", "Jocasta Ayrs", "Ovid", "Meronym"]	4																										
4 "Tom Hanks"	"Cloud Atlas"	["Zachry", "Dr. Henry Goose", "Isaac Sachs", "Dermot Hoggins"]	4																										
5 "Jim Broadbent"	"Cloud Atlas"	["Vyvyan Ayrs", "Captain Molyneux", "Timothy Cavendish"]	3																										
6 "Meg Ryan"	"Joe Versus the Volcano"	["DeDe", "Angelica Graynamore", "Patricia Graynamore"]	3																										
Utilidad en un sistema real	En gestión de proyectos, identificar quién ejerce como desarrollador, arquitecto y tester simultáneamente en un mismo sistema.																												

Análisis: a diferencia de las consultas anteriores que evalúan qué nodos están conectados y cómo, este se concentra en las propiedades de la relación. La relación ACTED_IN tiene un atributo roles que almacena una lista de personajes interpretados. El filtro `size(r.roles) > 1` evalúa directamente el tamaño de esa lista en cada relación individual, para no tener que recorrer nodos adicionales.

Esto es parecido a un filtro sobre una columna de tabla de hechos en un modelo relacional, pero accedido sin JOIN intermedio: la relación es simultáneamente la conexión entre nodos y el contenedor del dato de interés. A nivel de ejecución, Neo4j debe examinar todas las relaciones ACTED_IN del grafo para evaluar `size(r.roles)`. En un dataset de mayor escala, un índice sobre esta propiedad de relación podría mejorar el rendimiento aunque en Neo4j los índices de propiedades de relaciones tienen soporte limitado comparado con propiedades de nodos.

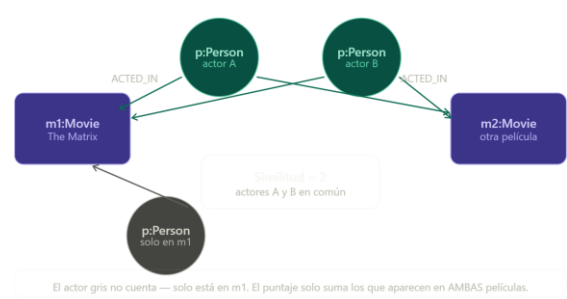


Q15 (Difícil)

Propósito	
-----------	--

Patrón de grafo utilizado	Se utilizó un patrón de traversal simétrico donde un nodo Person actúa como puente entre dos nodos Movie distintos. Desde cada película m1 se recorre una relación entrante ACTED_IN hacia un actor, y desde ese mismo actor se recorre una relación saliente ACTED_IN hacia una segunda película m2. El resultado captura todos los pares de películas conectadas por actores en común.																																																												
Resultado	<table><tr><th>Película1</th><th>Película2</th><th>ActoresCompartidos</th><th>Similitud</th></tr><tr><td>1 "The Matrix"</td><td>"The Matrix Reloaded"</td><td>["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]</td><td>4</td></tr><tr><td>2 "The Matrix Reloaded"</td><td>"The Matrix Revolutions"</td><td>["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]</td><td>4</td></tr><tr><td>3 "The Matrix"</td><td> "The Matrix Revolutions"</td><td>["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]</td><td>4</td></tr><tr><td>4 "A Few Good Men"</td><td>"Jerry Maguire"</td><td>["Cuba Gooding Jr.", "Tom Cruise"]</td><td>2</td></tr><tr><td>5 "A Few Good Men"</td><td>"As Good as It Gets"</td><td>["Jack Nicholson", "Cuba Gooding Jr."]</td><td>2</td></tr><tr><td>6 "You've Got Mail"</td><td>"Sleepless in Seattle"</td><td>["Meg Ryan", "Tom Hanks"]</td><td>2</td></tr><tr><td>7 "Sleepless in Seattle"</td><td>"Joe Versus the Volcano"</td><td>["Meg Ryan", "Tom Hanks"]</td><td>2</td></tr><tr><td>8 "You've Got Mail"</td><td>"Joe Versus the Volcano"</td><td>["Meg Ryan", "Tom Hanks"]</td><td>2</td></tr><tr><td>9 "Speed Racer"</td><td>"Ninja Assassin"</td><td>["Ben Miles", "Rain"]</td><td>2</td></tr><tr><td>10 "A Few Good Men"</td><td>"Hoffa"</td><td>["J.T. Walsh", "Jack Nicholson"]</td><td>2</td></tr><tr><td>11 "The Green Mile"</td><td>"Apollo 13"</td><td>["Tom Hanks", "Gary Sinise"]</td><td>2</td></tr><tr><td>12 "Hoffa"</td><td>"One Flew Over the Cuckoo's Nest"</td><td>["Jack Nicholson", "Danny DeVito"] </td><td>2</td></tr><tr><td>13 "Apollo 13"</td><td>"A League of Their Own"</td><td>["Bill Paxton", "Tom Hanks"]</td><td>2</td></tr><tr><td>14 "Sleepless in Seattle"</td><td>"A League of Their Own"</td><td>["Tom Hanks", "Rosie O'Donnell"]</td><td>2</td></tr></table>	Película1	Película2	ActoresCompartidos	Similitud	1 "The Matrix"	"The Matrix Reloaded"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4	2 "The Matrix Reloaded"	"The Matrix Revolutions"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4	3 "The Matrix"	 "The Matrix Revolutions"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4	4 "A Few Good Men"	"Jerry Maguire"	["Cuba Gooding Jr.", "Tom Cruise"]	2	5 "A Few Good Men"	"As Good as It Gets"	["Jack Nicholson", "Cuba Gooding Jr."]	2	6 "You've Got Mail"	"Sleepless in Seattle"	["Meg Ryan", "Tom Hanks"]	2	7 "Sleepless in Seattle"	"Joe Versus the Volcano"	["Meg Ryan", "Tom Hanks"]	2	8 "You've Got Mail"	"Joe Versus the Volcano"	["Meg Ryan", "Tom Hanks"]	2	9 "Speed Racer"	"Ninja Assassin"	["Ben Miles", "Rain"]	2	10 "A Few Good Men"	"Hoffa"	["J.T. Walsh", "Jack Nicholson"]	2	11 "The Green Mile"	"Apollo 13"	["Tom Hanks", "Gary Sinise"]	2	12 "Hoffa"	"One Flew Over the Cuckoo's Nest"	["Jack Nicholson", "Danny DeVito"] 	2	13 "Apollo 13"	"A League of Their Own"	["Bill Paxton", "Tom Hanks"]	2	14 "Sleepless in Seattle"	"A League of Their Own"	["Tom Hanks", "Rosie O'Donnell"]	2
Película1	Película2	ActoresCompartidos	Similitud																																																										
1 "The Matrix"	"The Matrix Reloaded"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4																																																										
2 "The Matrix Reloaded"	"The Matrix Revolutions"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4																																																										
3 "The Matrix"	 "The Matrix Revolutions"	["Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss", "Keanu Reeves"]	4																																																										
4 "A Few Good Men"	"Jerry Maguire"	["Cuba Gooding Jr.", "Tom Cruise"]	2																																																										
5 "A Few Good Men"	"As Good as It Gets"	["Jack Nicholson", "Cuba Gooding Jr."]	2																																																										
6 "You've Got Mail"	"Sleepless in Seattle"	["Meg Ryan", "Tom Hanks"]	2																																																										
7 "Sleepless in Seattle"	"Joe Versus the Volcano"	["Meg Ryan", "Tom Hanks"]	2																																																										
8 "You've Got Mail"	"Joe Versus the Volcano"	["Meg Ryan", "Tom Hanks"]	2																																																										
9 "Speed Racer"	"Ninja Assassin"	["Ben Miles", "Rain"]	2																																																										
10 "A Few Good Men"	"Hoffa"	["J.T. Walsh", "Jack Nicholson"]	2																																																										
11 "The Green Mile"	"Apollo 13"	["Tom Hanks", "Gary Sinise"]	2																																																										
12 "Hoffa"	"One Flew Over the Cuckoo's Nest"	["Jack Nicholson", "Danny DeVito"] 	2																																																										
13 "Apollo 13"	"A League of Their Own"	["Bill Paxton", "Tom Hanks"]	2																																																										
14 "Sleepless in Seattle"	"A League of Their Own"	["Tom Hanks", "Rosie O'Donnell"]	2																																																										
Utilidad en un sistema real	Aplicable en motores de recomendación de plataformas de streaming, por ejemplo, si un usuario vio m1, el sistema puede recomendar m2 basándose en que comparten elenco. A mayor puntaje de similitud, más fuerte es la recomendación.																																																												

Análisis: Al buscar actores que aparezcan en 2 películas al mismo tiempo estamos buscando actores compartidos, tal que entre más actores en común haya, más similitud existe entre ellos.

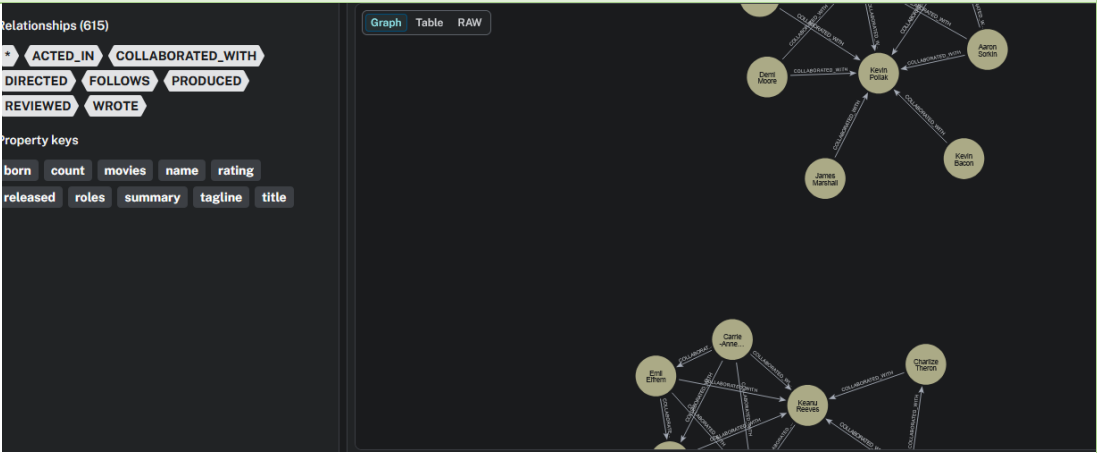


En términos más concretos, el patrón `(m1)<-[:ACTED_IN]-(p)-[:ACTED_IN]->(m2)` usa al nodo Person como puente entre dos nodos Movie. A nivel de grafo esto es un traversal de dos saltos

donde el nodo intermedio p es el elemento común. Neo4j lo resuelve encontrando todos los triángulos del tipo película-actor-película en el grafo.

Seguidamente el `WHERE id(m1) < id(m2)` es una condición de simetría que opera sobre los identificadores internos del grafo , debido a que el patrón no tiene dirección preferente entre m1 y m2, sin este filtro cada par aparece dos veces con los roles invertidos. Después el WITH crea un punto de corte en el pipeline donde Neo4j materializa los pares (m1, m2) con su lista de actores compartidos antes de continuar lo que permite que el `WHERE size(ActoresCompartidos) >= 2` filtre sobre la lista ya construida. Y finalmente, el `size()` sobre esa lista actúa como puntaje de similitud porque su valor es directamente proporcional al solapamiento de elenco entre los dos nodos Movie.

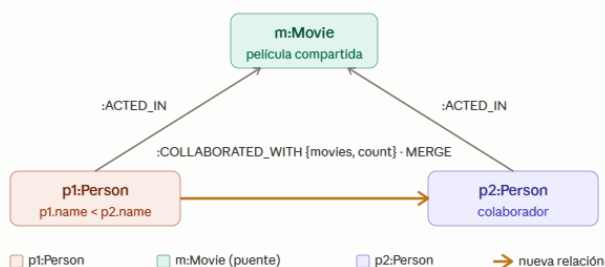
Q16 (Actualización)

Propósito	Crear una relación derivada :COLLABORATED_WITH entre personas que actuaron en una misma película
Patrón de grafo utilizado	(p1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(p2:Person) para encontrar a dos personas distintas que apunten a la misma película. WHERE p1.name < p2.name se usa para evitar duplicar resultados y que no aparezca la misma colaboración, pero con los nombres de los actores al revés, entonces esto hace que se ordene alfabéticamente. Luego con With y Merge se crea la relación solo si no existe. :COLLABORATED_WITH es la nueva relación
Resultado	
Utilidad en un sistema real	En algunas redes sociales se registra la información de los usuarios que estuvieron en el mismo lugar, con el fin de recomendar seguirse mutuamente. De esta manera, se guarda la información y no se tiene que estar recalculando cada vez que el usuario ve la sección de recomendaciones para seguir.

Análisis: este patrón lo que hace es usar algo como un "nodo puente compartido": dos nodos Person distintos (p1 y p2) apuntan hacia la misma película mediante :ACTED_IN. El WHERE p1.name < p2.name aplica un filtro de ordenamiento que garantiza que cada par aparezca una sola

vez en el resultado. Sin este filtro, tanto (Tom Hanks, Meg Ryan) como (Meg Ryan, Tom Hanks) serían filas válidas, duplicando cada colaboración.

La instrucción MERGE crea la relación `:COLLABORATED_WITH` solo si no existe entre ese par específico, lo que hace la consulta idempotente: puede ejecutarse múltiples veces sin crear duplicados. Esta técnica de materialización convierte en una arista directa $O(1)$ algo que antes requería un traversal de dos saltos en cada consulta. Internamente, Neo4j produce un producto cartesiano de todos los pares $(p1, p2)$ que comparten película y luego aplica `collect()` para agrupar las películas compartidas antes del MERGE, evitando crear una relación separada por cada película en común. Q17 aprovecha directamente esta relación materializada, lo que hace que el ranking de colaboraciones sea sustancialmente más rápido que recalcularlo desde los datos originales.



Q17 (Actualización)

Propósito	Listar las diez colaboraciones más fuertes según la cantidad de películas compartidas.																																
Patrón de grafo utilizado																																	
Resultado	<pre>neo4j\$ MATCH (p1:Person)-[:COLLABORATED_WITH]->(p2:Person) RETURN p1.name AS Actor1, p2.name AS Actor2, c.count AS PeliculasCompartidas, c.movies AS</pre><table><thead><tr><th>Actor1</th><th>Actor2</th><th>PeliculasCompartidas</th><th>ListaDePeliculas</th></tr></thead><tbody><tr><td>"Meg Ryan"</td><td>"Tom Hanks"</td><td>3</td><td>["You've Got Mail", "Sleepless in Seattle", "Joe Versus the Volcano"]</td></tr><tr><td>"Keanu Reeves"</td><td>"Laurence Fishburne"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr><tr><td>"Hugo Weaving"</td><td>"Laurence Fishburne"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr><tr><td>"Carrie-Anne Moss"</td><td>"Hugo Weaving"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr><tr><td>"Carrie-Anne Moss"</td><td>"Keanu Reeves"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr><tr><td>"Hugo Weaving"</td><td>"Keanu Reeves"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr><tr><td>"Carrie-Anne Moss"</td><td>"Laurence Fishburne"</td><td>3</td><td>["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]</td></tr></tbody></table>	Actor1	Actor2	PeliculasCompartidas	ListaDePeliculas	"Meg Ryan"	"Tom Hanks"	3	["You've Got Mail", "Sleepless in Seattle", "Joe Versus the Volcano"]	"Keanu Reeves"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]	"Hugo Weaving"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]	"Carrie-Anne Moss"	"Hugo Weaving"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]	"Carrie-Anne Moss"	"Keanu Reeves"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]	"Hugo Weaving"	"Keanu Reeves"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]	"Carrie-Anne Moss"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]
Actor1	Actor2	PeliculasCompartidas	ListaDePeliculas																														
"Meg Ryan"	"Tom Hanks"	3	["You've Got Mail", "Sleepless in Seattle", "Joe Versus the Volcano"]																														
"Keanu Reeves"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
"Hugo Weaving"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
"Carrie-Anne Moss"	"Hugo Weaving"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
"Carrie-Anne Moss"	"Keanu Reeves"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
"Hugo Weaving"	"Keanu Reeves"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
"Carrie-Anne Moss"	"Laurence Fishburne"	3	["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"]																														
Utilidad en un sistema real	Usar la relación directa resulta mucho menos costoso que estar calculando los rankings manualmente cada vez que se consultan.																																

Q18 (Análisis de Ejecución)

Consulta	Q7 Media-Alta
Ejecución EXPLAIN	<pre> graph TD A[NodeByLabelScan@neo4j m m:Movie Ordered by m ASC 38 estimated rows] --> B[Expand(All)@neo4j m, anon_0, p (m)->[anon_0:ACTED_IN]-(p) Ordered by m ASC 172 estimated rows] B --> C[Filter@neo4j m, anon_0, p p:Person Ordered by m ASC 172 estimated rows] C --> D[OrderedAggregation@neo4j m, Actores m, collect(p.name) AS Actores Ordered by m ASC 13 estimated rows] D --> E[Filter@neo4j m, Actores size(Actores) > \$autoint_0 Ordered by m ASC 4 estimated rows] E --> F[CacheProperties@neo4j m, Actores cache[m.title], cache[m.released] Ordered by m ASC 4 estimated rows] F --> G[Projection@neo4j Año, Actores, m, NúmeroDeActores, Pelicula cache[m.title] AS Pelicula, cache[m.released] AS Año, size(Actores) AS NúmeroDeActores Ordered by m ASC 4 estimated rows] G --> H[ProduceResults@neo4j Año, Actores, m, NúmeroDeActores, Pelicula Pelicula, Año, NúmeroDeActores, Actores Ordered by m ASC 4 estimated rows] H --> I[Result] </pre> Cypher version: 5, Planner: COST, Runtime: SLOTTED Total database accesses: 0, Total allocated memory: -, Total

Resumen texto

Planner COST

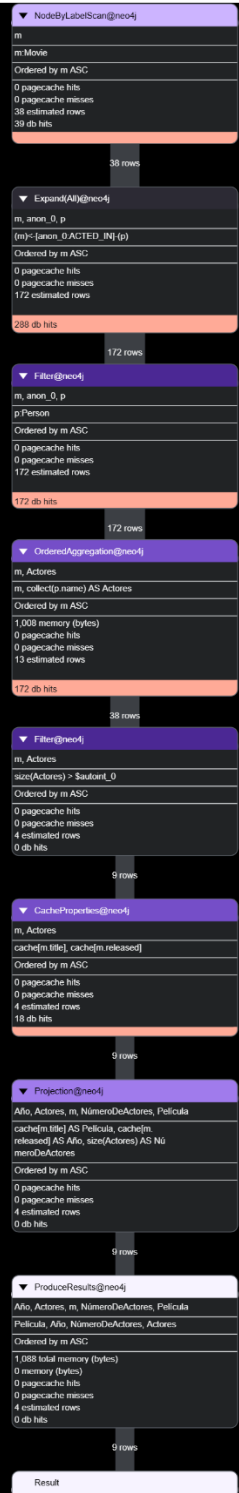
Runtime SLOTTED

Runtime version 3.26.26

Operator	Id	Details	Estimated Rows	Ordered by
•ProduceResults	0	Película, Año, NúmeroDeActores, Actores	4	
•Projection	1	cache[m.title] AS Película, cache[m.released] AS Año, size(Actores) AS NúmeroDeActores	4	
•CacheProperties	2	cache[m.title], cache[m.released]	4	
•Filter	3	size(Actores) > \$autoint_0	4	
•OrderedAggregation	4	m, collect(p.name) AS Actores	13	
•Filter	5	p:Person	172	
•Expand(All)	6	(m)←[anon_0:ACTED_IN]-(p)	172	
•NodeByLabelScan	7	m:Movie	38	m ASC

Total database accesses: ?

Ejecución PROFILE



Cypher version: 5, Planner: COST, Runtime: SLOTTED
Total database accesses: 689, Total allocated memory: 1,088 bytes, Total time: 112 ms

Resumen texto

	Planner COST Runtime SLOTTED Runtime version 5.26.26 <table><tr><th>Operator</th><th>Id</th><th>Details</th><th>Estimated Rows</th><th>Rows</th><th>DB Hits</th><th>Memory (Bytes)</th><th>Page Cache Hits/Misses</th><th>Ordered by</th></tr><tr><td>+ProduceResults</td><td>0</td><td>Película, Año, NúmeroDeActores, Actores</td><td>4</td><td>9</td><td>0</td><td>0</td><td>0/0</td><td></td></tr><tr><td>+Projection</td><td>1</td><td>cache[m.title] AS Película, cache[m.released] AS Año, size(Actores) AS NúmeroDeActores</td><td>4</td><td>9</td><td>0</td><td></td><td>0/0</td><td></td></tr><tr><td>+CacheProperties</td><td>2</td><td>cache[m.title], cache[m.released]</td><td>4</td><td>9</td><td>18</td><td></td><td>0/0</td><td></td></tr><tr><td>+Filter</td><td>3</td><td>size(Actores) > \$autoint_0</td><td>4</td><td>9</td><td>0</td><td></td><td>0/0</td><td></td></tr><tr><td>+OrderedAggregation</td><td>4</td><td>m, collect(p.name) AS Actores</td><td>13</td><td>38</td><td>172</td><td>1008</td><td>0/0</td><td></td></tr><tr><td>+Filter</td><td>5</td><td>p:Person</td><td>172</td><td>172</td><td>172</td><td></td><td>0/0</td><td></td></tr><tr><td>+Expand(All)</td><td>6</td><td>(n)-[anon_0:ACTED_IN]-(p)</td><td>172</td><td>172</td><td>288</td><td></td><td>0/0</td><td></td></tr><tr><td>+NodeByLabelScan</td><td>7</td><td>m:Movie</td><td>38</td><td>38</td><td>39</td><td></td><td>0/0</td><td>m ASC</td></tr></table> Total database accesses: 689, total allocated memory: 1808	Operator	Id	Details	Estimated Rows	Rows	DB Hits	Memory (Bytes)	Page Cache Hits/Misses	Ordered by	+ProduceResults	0	Película, Año, NúmeroDeActores, Actores	4	9	0	0	0/0		+Projection	1	cache[m.title] AS Película, cache[m.released] AS Año, size(Actores) AS NúmeroDeActores	4	9	0		0/0		+CacheProperties	2	cache[m.title], cache[m.released]	4	9	18		0/0		+Filter	3	size(Actores) > \$autoint_0	4	9	0		0/0		+OrderedAggregation	4	m, collect(p.name) AS Actores	13	38	172	1008	0/0		+Filter	5	p:Person	172	172	172		0/0		+Expand(All)	6	(n)-[anon_0:ACTED_IN]-(p)	172	172	288		0/0		+NodeByLabelScan	7	m:Movie	38	38	39		0/0	m ASC	
Operator	Id	Details	Estimated Rows	Rows	DB Hits	Memory (Bytes)	Page Cache Hits/Misses	Ordered by																																																																											
+ProduceResults	0	Película, Año, NúmeroDeActores, Actores	4	9	0	0	0/0																																																																												
+Projection	1	cache[m.title] AS Película, cache[m.released] AS Año, size(Actores) AS NúmeroDeActores	4	9	0		0/0																																																																												
+CacheProperties	2	cache[m.title], cache[m.released]	4	9	18		0/0																																																																												
+Filter	3	size(Actores) > \$autoint_0	4	9	0		0/0																																																																												
+OrderedAggregation	4	m, collect(p.name) AS Actores	13	38	172	1008	0/0																																																																												
+Filter	5	p:Person	172	172	172		0/0																																																																												
+Expand(All)	6	(n)-[anon_0:ACTED_IN]-(p)	172	172	288		0/0																																																																												
+NodeByLabelScan	7	m:Movie	38	38	39		0/0	m ASC																																																																											
Operadores en Neo4j	NodeByLabelScan: escanea todos los nodos con la etiqueta Movie en el grafo. Este funciona como el punto de entrada ya que Neo4j no tiene otra forma de empezar porque no hay índice por el que filtrar. Expand(All): desde cada nodo Movie recorre todas las relaciones ACTED_IN entrantes para alcanzar los nodos Person. Es el traversal del patrón de grafo. Filter: verifica que los nodos alcanzados sean efectivamente de tipo Person. Aunque el patrón lo implica, Neo4j lo confirma explícitamente aquí. OrderedAggregation: ejecuta el collect(p.name) agrupando los actores por película. Es donde el WITH materializa la lista Actores. Filter (segundo) : aplica el WHERE size(Actores) > 5 sobre las listas ya construidas. Descarta las películas con pocos actores. CacheProperties: precarga en memoria las propiedades m.title y m.released que se van a necesitar en la proyección, evitando ir a disco por cada fila. Projection: construye las columnas finales del RETURN renombrando y calculando size(Actores). ProduceResults: entrega el resultado final al cliente.																																																																																		
Filas procesadas	<table><tr><th>Operador</th><th>EXPLAIN (estimado)</th><th>PROFILE (real)</th></tr><tr><td>NodeByLabelScan</td><td>38 filas</td><td>38 filas</td></tr><tr><td>Expand(All)</td><td>172 filas</td><td>172 filas</td></tr><tr><td>Filter (Person)</td><td>172 filas</td><td>172 filas</td></tr><tr><td>OrderedAggregation</td><td>13 filas</td><td>38 filas</td></tr><tr><td>Filter (size > 5)</td><td>4 filas</td><td>9 filas</td></tr><tr><td>CacheProperties</td><td>4 filas</td><td>9 filas</td></tr><tr><td>Projection</td><td>4 filas</td><td>9 filas</td></tr><tr><td>ProduceResults</td><td>4 filas</td><td>9 filas</td></tr><tr><td>TOTAL</td><td>411 filas</td><td>456 filas</td></tr></table>	Operador	EXPLAIN (estimado)	PROFILE (real)	NodeByLabelScan	38 filas	38 filas	Expand(All)	172 filas	172 filas	Filter (Person)	172 filas	172 filas	OrderedAggregation	13 filas	38 filas	Filter (size > 5)	4 filas	9 filas	CacheProperties	4 filas	9 filas	Projection	4 filas	9 filas	ProduceResults	4 filas	9 filas	TOTAL	411 filas	456 filas																																																				
Operador	EXPLAIN (estimado)	PROFILE (real)																																																																																	
NodeByLabelScan	38 filas	38 filas																																																																																	
Expand(All)	172 filas	172 filas																																																																																	
Filter (Person)	172 filas	172 filas																																																																																	
OrderedAggregation	13 filas	38 filas																																																																																	
Filter (size > 5)	4 filas	9 filas																																																																																	
CacheProperties	4 filas	9 filas																																																																																	
Projection	4 filas	9 filas																																																																																	
ProduceResults	4 filas	9 filas																																																																																	
TOTAL	411 filas	456 filas																																																																																	
Interpretación	La diferencia más notable entre los planes de ejecución de EXPLAIN y de PROFILE la vemos en OrderedAggregation. Esto pasa porque EXPLAIN es una función que ve por encima la estructura de el query si establece el orden de ejecución sin realmente ejecutar recursos. Mientras que PROFILE si ejecuta el query, lo que impone un gasto de recursos y obtención de métricas reales. Lo que sucedió en este query fue que EXPLAIN estimo 13 agrupaciones mientras que las reales encontradas fueron 38 según PROFILE, lo que demuestra que se subestimo la distribución real de actores por película con EXPLAIN.																																																																																		

	En cuanto al costo, esta consulta no es costosa porque el dataset es ya muy pequeño con apenas 38 nodos Movie, y los operadores WITH + WHERE usados en el query se encargan de descartar filas temprano en el pipeline. Sin embargo, un punto débil puede ser el NodeByLabelScan donde Neo4j va a recorrer todas las películas sin excepción debido a la ausencia de índices o filtros de entrada.
Propuesta de Mejora	Para mejorar esta consulta se deberían de agregar índices sobre movie, o filtrar por alguna propiedad indexada desde el MATCH para reducir los nodos que debe recorrer.

Consulta

Ejecución EXPLAIN

▼ NodeByLabelScan@neo4j

m

m:Movie

38 estimated rows

▼ CacheByProperty@neo4j

m

cacheBy: title

38 estimated rows

▼ ExpandAll@neo4j

m, t, a

(m)->ACTED_IN(a)

1/2 estimated rows

▼ Filter@neo4j

m, t, a

size(cacheBy: title) > 500000 AND a: Person

52 estimated rows

▼ Prune@neo4j

a: m, Actor, CantidadDeRoles, r

size(cacheBy: title) AS CantidadDeRoles, a: name AS Actor

52 estimated rows

▼ Sort@neo4j

a: m, Actor, CantidadDeRoles, r

CantidadDeRoles DESC, Actor ASC

Ordered by CantidadDeRoles DESC, Actor ASC

52 estimated rows

▼ Projection@neo4j

listOfRoles, a: m, Actor, CantidadDeRoles, r: Pelicula

cacheBy: title AS Pelicula, cacheBy: title AS listOfRoles

Ordered by CantidadDeRoles DESC, Actor ASC

52 estimated rows

▼ Projection@neo4j

listOfRoles, a: m, Actor, CantidadDeRoles, r: Pelicula

Actor, Pelicula, listOfRoles, CantidadDeRoles

Ordered by CantidadDeRoles DESC, Actor ASC

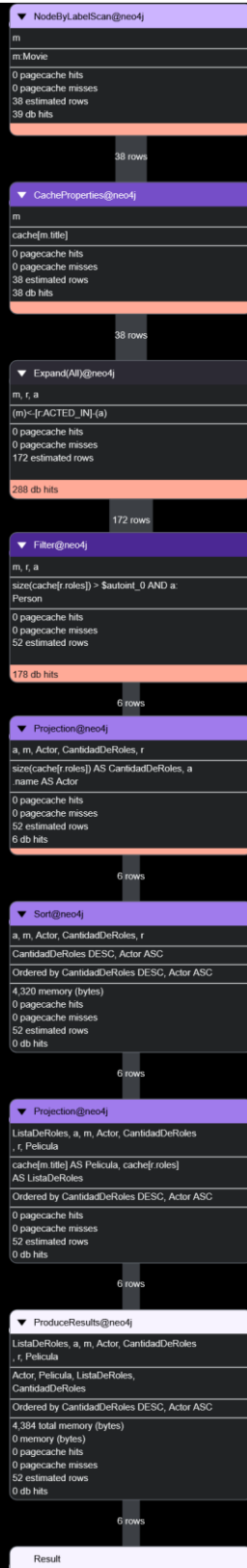
52 estimated rows

Result

Cypher version: 5, Planner: COST, Runtime: SLOTTED

Total database accesses: 0, Total allocated memory: -, Total time: 1,966 ms

Ejecución PROFILE



Cypher version: 5, Planner: COST, Runtime: SLOTTED
Total database accesses: 549, Total allocated memory: 4,384 bytes, Total time: 362 ms

Plan de ejecución	Comienza buscando el nodo inicial, escaneando todos los nodos de una etiqueta (por ejemplo, Person) para iniciar el recorrido, ya que no hay un nodo ancla (como un WHERE a.name = "Keanu").																																
Operadores en Neo4j	NodeByLabelScan: Escanea todos los nodos de una etiqueta (por ejemplo, Person) para iniciar el recorrido, ya que no hay un nodo ancla (como un WHERE a.name = "Keanu"). Expand(All): Recorre todas las relaciones ACTED_IN que salen de los nodos iniciales. Filter: Evalúa la condición size(r.roles) > 1 sobre las relaciones encontradas. Projection: Construye las columnas que se solicitaron en el RETURN (nombre, título, lista de roles y la cantidad). Sort: Ordena en memoria los resultados descendente por cantidad y ascendente por nombre del actor. ProduceResults: Entrega la salida final al usuario.																																
Filas procesadas	<table><tr><th>Operador</th><th>EXPLAIN</th><th>PROFILE</th></tr><tr><td>NodeByLabelScan</td><td>38</td><td>38</td></tr><tr><td>CacheProperties</td><td>38</td><td>38</td></tr><tr><td>Expand(All)</td><td>172</td><td>172</td></tr><tr><td>Filter</td><td>52</td><td>6</td></tr><tr><td>Projection</td><td>52</td><td>6</td></tr><tr><td>Sort</td><td>52</td><td>6</td></tr><tr><td>Projection</td><td>52</td><td>6</td></tr><tr><td>ProduceResults</td><td>52</td><td>6</td></tr><tr><td>TOTAL</td><td>508</td><td>278</td></tr></table>			Operador	EXPLAIN	PROFILE	NodeByLabelScan	38	38	CacheProperties	38	38	Expand(All)	172	172	Filter	52	6	Projection	52	6	Sort	52	6	Projection	52	6	ProduceResults	52	6	TOTAL	508	278
Operador	EXPLAIN	PROFILE																															
NodeByLabelScan	38	38																															
CacheProperties	38	38																															
Expand(All)	172	172																															
Filter	52	6																															
Projection	52	6																															
Sort	52	6																															
Projection	52	6																															
ProduceResults	52	6																															
TOTAL	508	278																															
Interpretación	La diferencia más notable entre los planes de ejecución se encuentra a partir del operador Filter. Neo4j estimó estadísticamente (EXPLAIN) que de las 172 relaiones evaluadas, 52 cumplirían con tener un tamaño de roles mayor a 1. Sin embargo, al ejecutar la consulta realmente (PROFILE), se sucedió que el filtro es muchísimo más estricto, ya que solo 6 relaciones (filas reales) cumplieron con la condición de múltiples roles. Esta sobreestimación del EXPLAIN provocó que creyera que los operadores siguientes (Projection, Sort y ProduceResults) tendrían que procesar 52 filas cada uno, cuando en la realidad solo procesaron 6. El PROFILE también revela que el cuello de botella físico de esta consulta está en el Expand(All), el cual generó 288 db hits (accesos a la base de																																

	datos) al tener que buscar todas las relaciones de actuación disponibles antes de poder filtrarlas."
Propuesta de Mejora	Calcular <code>size()</code> acaba siendo muy costoso porque en Neo4j las propiedades de las relaciones no se indexan tan eficientemente como las de los nodos. Lo ideal es una modificación en el modelo de datos. Entonces, durante la carga de datos se podría agregar una propiedad numérica entera directamente en la relación llamada <code>r.roles_count</code> . De esta manera, el filtro sería un simple <code>WHERE r.roles_count > 1</code> , evitando usar la función <code>size()</code> sobre arreglos dinámicos en tiempo de consulta. Alternativamente, se podrían materializar los roles como nodos independientes (Actor -> INTERPRETO -> Rol -> EN -> Pelicula), lo que permitiría usar patrones topológicos que son mucho más rápidos para Neo4j que leer propiedades.

Conclusiones técnicas

El desarrollo de esta práctica permitió explorar las capacidades de Neo4j como sistema de gestión de bases de datos orientado a grafos, y los resultados son bastante claros en cuanto a sus fortalezas.

La primera conclusión es sobre la naturaleza del modelo. En una base de datos relacional, representar múltiples niveles de relaciones entre entidades implica cadenas de JOIN que crecen en complejidad conforme se profundiza en el modelo. En Neo4j esa misma lógica se expresa como un patrón de transversal directo, donde la relación no es un mecanismo auxiliar para conectar tablas sino el dato en sí mismo. Esto no es solo una ventaja de sintaxis, sino que constituye una diferencia estructural que impacta directamente en la legibilidad y el mantenimiento de las consultas.

La segunda conclusión tiene que ver con la potencia de las consultas indirectas. Uno de los hallazgos más interesantes fue la capacidad de explorar conexiones transitivas con profundidad variable, es decir, no solo consultar la relación directa entre A y B, sino extenderse a los vecinos de B para encontrar conexiones relevantes para A. Esto es la base de sistemas de recomendación, análisis de redes sociales y detección de caminos, y en Cypher se expresa de forma natural sin reescribir la lógica de la consulta.

Finalmente, vale reconocer que los grafos no son una solución universal. Para datos altamente tabulares o agregaciones masivas el modelo relacional sigue siendo más eficiente. Sin embargo, los grafos brillan específicamente cuando las relaciones entre los datos son tan importantes como los datos mismos.